

Software Design

Chapter 5 — Source-Grounded Study Notes

Course: Advanced Software Engineering (CS603), 3 Credit Hours

Instructor: Asst. Prof. Dr. Ali Fahim Ni'ma

Institution: College of Computer Science & Information Technology, University of Wasit

Source: Aggarwal & Singh, Software Engineering, 3rd ed., Ch.5 companion slides (99 slides)

Method: Every paragraph anchored to its source slide as [S:n]

Status: Draft for review — verify anchors before final use

Table of Contents

Table of Contents.....	2
List of Figures.....	3
5. Software Design.....	4
5.1 What Is Design?.....	4
5.1.1 Design and Analysis	4
5.1.2 The Design Process and Its Products	5
5.1.3 The Design Framework.....	5
5.1.4 High-Level and Detailed Design.....	6
5.1.5 Design Methodologies	6
5.1.6 Design Principles and Concepts.....	6
5.1.7 Conceptual and Technical Designs	7
5.1.8 Objectives of Design.....	8
5.1.9 Why Design Is Important.....	9
5.2 Modularity	10
5.2.1 Modularity and the Modular System	10
5.2.2 Modularity and Software Cost	11
5.2.3 Module Coupling	12
5.2.4 Module Cohesion.....	15
5.2.5 The Relationship between Cohesion and Coupling	17
5.2.6 Layered Arrangement of Modules	17
5.3 Strategy of Design	18
5.3.1 The Need for a Design Strategy	18
5.3.2 Bottom-Up Design.....	19
5.3.3 Top-Down Design.....	20
5.3.4 Hybrid Design.....	20
5.3.5 Choosing among the Strategies.....	21

(In Word: select all and press F9, or right-click > Update Field.)

List of Figures

Figure 5.1. The design framework.

Figure 5.2. A two-part design process.

Figure 5.3. The transformation of an informal design into a detailed design.

Figure 5.4. Modularity and software cost.

Figure 5.5. Degrees of module coupling: uncoupled, loosely coupled, and highly coupled.

Figure 5.6. Example of coupling: editing a student record, tight versus loose.

Figure 5.7. The types of module coupling, from best (data) to worst (content).

Figure 5.8. Example of common coupling through a shared global data area.

Figure 5.9. The types of module cohesion, from weakest (coincidental) to strongest (functional).

Figure 5.10. A view of cohesion and coupling: high coupling (left) versus low coupling (right).

Figure 5.11. Bottom-up tree structure: library modules combine upward into larger modules.

Figure 5.12. Top-down structure (left) versus the design-reusable structure (right).

5. Software Design

5.1 What Is Design?

The design phase is the stage of software development in which the designer plans *how* a software system is to be produced so that it is functional, reliable, and reasonably easy to understand, modify, and maintain. Its input is the software requirements specification (SRS), which states *what* the system must do; its output is a software design document (SDD), which states *how* the system will do it. Designing a software system therefore means determining how the requirements are to be realised, and the purpose of the design phase can be stated in one sentence: to produce a working solution to the problem posed in the requirements specification. Design is the critical link between requirements engineering and implementation, because it is the activity that identifies the main structural components of the system and the relationships between them.

Design is a more creative activity than analysis, and this creativity has a consequence that surprises newcomers: there is no such thing as a single *correct* design. Analysis can, in principle, arrive at a correct model of an existing system, but good design is always system-dependent: what is good design for one system may be bad for another. Design is a problem-solving activity and, in practice, very much a matter of trial and error; the designer works together with the users, searching for a solution with professional judgement until there is agreement that a satisfactory solution has been found.

Why give design a phase of its own? The answer is scale. For a small project, such as a student exercise, one can sit with the specification and simply write the program. For a larger project, a wide gap opens between the specification and the code, and that gap has to be bridged with something more concrete. The bridge is the software design.

5.1.1 Design and Analysis

Design and analysis differ in both goal and scope, and the difference is worth stating precisely because the two are often confused.

The goal of analysis is to elaborate the customer's requirements through careful thinking, while deliberately avoiding any decision about the exact way the system will be implemented. An analysis model is therefore generic: it does not consider implementation, platforms, or other technology-specific issues, and it is normally very difficult to implement directly in a programming language. The goal of design is the opposite. Design takes the analysis model and introduces the decisions that fix the way the system will be built, producing a model that reflects those decisions and is detailed enough to be implemented with little further interpretation. In short, design is the transformation of the analysis model, over a series of steps, into a design model.

The two models are also documented with different notations, and this is a useful practical signal of which phase one is in. In the function-oriented approach the analysis model is documented using data-flow diagrams (DFDs) while the design is documented using structure charts; in the object-oriented approach both the analysis and the design models are documented using the Unified Modeling Language (UML), through different families of diagrams. A model that still reads as DFDs has not yet crossed into design.

A further difference from analysis is that, even when the same methodology is used, different designers usually arrive at very different designs for the same problem. The reason is that a design technique requires the designer to make many subjective decisions and to work out compromises between contradictory objectives. It follows that even a single designer can produce several different solutions to the same problem, and that obtaining a good design involves trying out several candidate solutions and selecting the best one.

A natural question at this point is whether writing a program is itself an act of design. After all, a programmer decides how a function will be structured while writing it. The distinction the literature draws is one of *level* and *intent*. Coding is concerned with expressing a solution in the syntax of a particular programming language; design is concerned with the solution's structure, decomposition, interfaces, and algorithms, before and independently of any specific language. Design deliberately postpones the language so that the structural decisions can be made on their own merits, and so that the same design could, if necessary, be implemented in a different language. When a programmer does make structural decisions while coding, they are in fact designing, though designing informally, and without the review and documentation that a dedicated design phase provides. The reason large projects separate the two activities is precisely that informal, undocumented design decisions, made under the pressure of producing code, are the decisions most likely to be wrong and most expensive to reverse.

5.1.2 The Design Process and Its Products

The design process transforms the SRS document into the design document. The design document produced at the end of the design phase must be implementable using a programming language in the subsequent coding phase, and it is normally reviewed by the members of the development team to ensure that the design solution conforms to the requirements specification.

The following items are designed and documented during the design phase. First, the different **modules** required by the solution are identified; each module is a collection of functions together with the data shared by those functions, each module should accomplish some well-defined task within the overall responsibility of the software, and each module should be named according to the task it performs. Second, the **control relationships** among the modules, which arise from function calls across modules, are identified and recorded. Third, the **interfaces** among the modules are specified, and an interface fixes the exact data items exchanged when one module invokes a function of another. Fourth, the **data structures** of the individual modules are designed; each module usually stores data that its functions must share, and suitable structures for storing and managing that data must be designed and documented. Finally, the **algorithms** required to implement the individual modules are designed and documented, with due consideration for the accuracy of the results and for their space and time complexity.

From a wider view, the inputs to the design effort are an understanding of the requirements, the environmental constraints, and the design criteria; and its outputs are the architecture design, which shows how the pieces are interrelated, the specifications for any new pieces, and the definitions for any new data.

5.1.3 The Design Framework

Design proceeds through a framework of activities rather than in a single step (Figure 5.1). The framework starts from the initial requirements and ends with the completed design. Data is gathered on the user requirements and analysed; the questions raised by the requirements are answered, and a high-level design is then conceived. The design is refined and documented, and it is validated against the requirements on a regular basis. Because validation can raise fresh requirement questions, the activity is cyclic: the design is refined in every pass and finally documented as the software design document. The cyclic nature is not a defect of the framework but its essence: it is how the designer reconciles the requirements with the solution before committing to code.

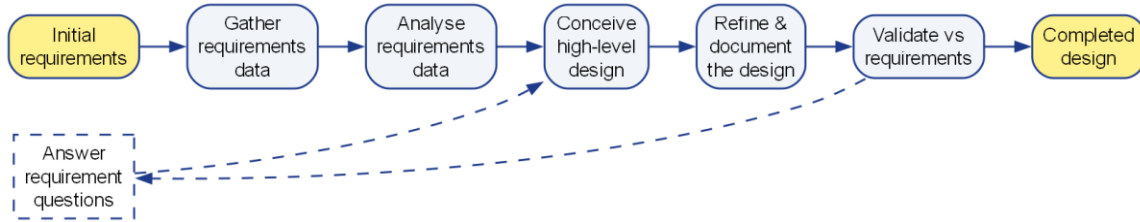


Figure 5.1. The design framework.

5.1.4 High-Level and Detailed Design

Whatever methodology is used, the design activities are broadly classified into two stages: **preliminary (or high-level) design**, and **detailed design**.

In high-level design, the problem is decomposed into a set of modules; the control relationships among the modules are identified, and the interfaces among the modules are identified. The outcome of high-level design is called the program structure, or the software architecture. High-level design is the crucial step of the whole design: when it is complete, the problem should have been decomposed into many small, functionally independent modules that are cohesive, have low coupling among themselves, and are arranged in a hierarchy. A tree-like diagram called the structure chart is widely used to represent a high-level design for procedural development, while UML diagrams are used to document an object-oriented high-level design.

Once the high-level design is complete, detailed design is undertaken. During detailed design each module is examined carefully to design its data structures and its algorithms. The outcome of the detailed design stage is usually documented as a module specification (MSPEC), which describes the data structures and algorithms of each module precisely enough for programmers to begin coding.

5.1.5 Design Methodologies

The design activities vary considerably with the design methodology being used. A large number of methodologies exist, but they can be roughly classified into two approaches: the procedural (function-oriented) approach and the object-oriented approach. These are two fundamentally different paradigms. They organise a system around two different centres of gravity: the function-oriented approach decomposes the design into interacting units each with a clearly defined function, whereas the object-oriented approach focuses attention not on the functions performed by the program but on the data that are to be manipulated. Both paradigms are treated in the later sections of this chapter.

5.1.6 Design Principles and Concepts

Running beneath all of the methodologies is a small set of design principles that explain *why* the methods work. The most important are abstraction, refinement, modularity, and information hiding.

Abstraction is the elimination of the irrelevant and the amplification of the essentials. Designers work with three forms of abstraction: procedural abstraction, in which a named procedure is described by its purpose without specifying its internal details; data abstraction, in which a named data object is described without specifying its internal representation; and control abstraction, in which a program-control mechanism is implied without specifying its internal details.

Refinement is the complementary concept. Stepwise refinement is a top-down strategy in which a program is developed by successively refining levels of procedural detail: a hierarchy is developed by decomposing a macroscopic statement of function, in a stepwise fashion, until programming-language statements are reached. Each refinement step implies design decisions, so the designer must be aware of the criteria for those decisions

and of the existence of alternative solutions. Abstraction and refinement are two sides of one process: abstraction lets the designer suppress low-level detail, while refinement reveals it as the design progresses. Together they let a complete design model be built gradually.

Modularity is the decomposition of a system into separately named and addressable components, called modules, that are integrated to satisfy the requirements. Its power comes from a "divide and conquer" argument: it is easier to solve a complex problem when it can be broken into manageable pieces. Its risk is over-modularisation, treated in Section 5.2 through the relationship between the number of modules and the cost of the system.

Information hiding is the principle that each module should hide its internal decisions from the rest of the system, exposing only what others need. It is the mechanism that makes modules replaceable without disturbing the rest of the design, and it reappears as *encapsulation* in object-oriented design (Section 5.6). A related principle, **problem partitioning**, is the decomposition of the problem itself into sub-problems so that the overall complexity is reduced.

5.1.7 Conceptual and Technical Designs

The process of design is the transformation of ideas into detailed implementation descriptions, with the goal of satisfying the software requirements. To transform requirements into a working system, the designer must satisfy two very different audiences. The customers understand *what* the system is to do, while the system builders (the programmers who write the code) must understand *how* the system is to work. Because these two audiences need two different descriptions of the same system, design is a two-part, iterative process.

The first part is the **conceptual design**, which tells the customer exactly what the system will do. Once the customer has approved the conceptual design, it is translated into a much more detailed document, the **technical design**, which allows the system builders to understand the actual hardware and software needed to solve the customer's problem. Figure 5.2 shows this two-part process.

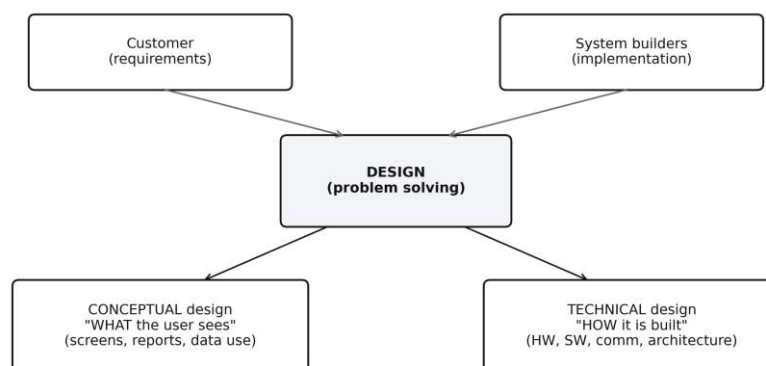


Figure 5.2. A two-part design process.

The two documents describe the same system, but in different ways, because they are written for different audiences. The conceptual design answers the following questions:

- Where will the data come from?
- What will happen to the data in the system?
- How will the system look to users?
- What choices will be offered to users?

- What is the timing of events?
- How will the reports and screens look like?

The conceptual design describes the system in a language understandable to the customer. It contains no technical jargon and is independent of any implementation. The technical design, by contrast, describes the hardware configuration, the software needs, the communication interfaces, the input and output of the system, the network architecture, and anything else that translates the requirements into a solution to the customer's problem.

The relationship between the two parts is best understood as a contract. The customer approves the *what*; the builders then own the *how*. Keeping the two separate means that the customer is never asked to validate technical decisions they cannot judge, and the builders are never left to infer the customer's intent from an abstract statement. The two-part split is not always necessary, however: sometimes the customers are themselves software developers, and are sophisticated enough to understand the *what* and the *how* together. In such a case a single, comprehensive design document may be produced instead of two separate ones.

5.1.8 Objectives of Design

The specification, the "outside" view of a program, should be as free as possible of the aspects imposed by *how* the program will work, that is, the "inside" view. It is seldom a document from which coding can be done directly. Design therefore fills the gap between the specification and the coding: it takes the specification, decides how the program will be organized and what methods it will use, and expresses this in sufficient detail to be directly codeable. When the specification calls for a large or complex program, the design is likely to work down through a number of levels, at each level breaking the implementation problem into smaller and simpler problems. Filling a large gap involves a number of stepping-stones, and the wider the gap, the more stepping-stones are needed.

Whatever its size, the design must satisfy a set of objectives. The four most frequently quoted are:

- **Correct and complete.** A design must faithfully implement all of the functionality required by the specification, and it must cover all of the relevant data structures, modules, external interfaces, and module interconnections.
- **Understandable.** A design that cannot be understood cannot be implemented or maintained with confidence; understandability is discussed further in Section 5.1.9.
- **At the right level.** A design must be expressed at a level of abstraction appropriate both to the customer who validates it and to the programmers who build from it: neither so coarse that it leaves decisions unexplained, nor so fine that it is merely code in another form.
- **Maintainable.** Because change requests continue to arrive long after the product is released, and because maintenance consumes the largest share of a product's lifetime cost, a design must facilitate the maintenance of the code produced from it.

A fuller account extends this list. **Efficiency** requires the design to use scarce resources (processor time and memory in particular) sensibly, because these resources are costly. **Verifiability** requires that the correctness of the design can be checked, so that verification techniques can be applied easily. **Completeness** requires that every component of the design is verified and specified. **Traceability** requires that every element of the design can be traced back to a requirement, which is what makes verification possible in the first place. **Simplicity** is sometimes named as the single most important quality, on the grounds that a simple design is the one most easily maintained.

The full set of objectives, gathered from the standard treatments, is summarized below. They are not alternatives to one another; a good design is expected to satisfy all of them as far as the application permits, and the skill of design lies in balancing them when they pull in different directions.

Objective	What it requires	Why it matters
Correctness	The design faithfully implements every required function	An incorrect design cannot yield a correct system
Completeness	All data structures, modules, interfaces and interconnections are specified	Nothing is left for the coder to invent
Verifiability	The correctness of the design can be checked	Design reviews and formal checks become possible
Traceability	Every design element can be traced to a requirement	Requirements are demonstrably covered; scope creep is visible
Understandability	The design is graspable without excessive effort	A design that overwhelms the reader leads to defects and cost
Efficiency	Processor time, memory and cost are used sensibly	Scarce resources are expensive
Simplicity	The structure is as uncomplicated as the problem allows	Simplicity is the foundation of maintainability
Maintainability	The design can be changed easily and safely	Most of a product's lifetime cost is spent on change

It is worth noting, as a matter of professional judgement rather than of fact, that the different authorities do not agree on which objective matters most. One widely used text argues that understandability is the most important issue when judging a design, because a design that overwhelms the reader's mind leads to defects and runaway cost. Another argues that simplicity is the most important quality criterion, for much the same reason. The disagreement is itself instructive: the objectives are not a checklist to be ticked but a set of tensions to be balanced, and the balance depends on the application. For an embedded system with tightly constrained memory, for example, design comprehensibility may be consciously sacrificed to achieve code compactness; for a general business system the opposite trade-off is usual.

Designers do not arrive at a finished design document immediately. The design is developed iteratively through a number of different phases, adding detail as it grows while constantly backtracking to correct earlier, less formal versions. The starting point is an informal design outline, which is refined by adding information to make it consistent and complete, passing through progressively more formal stages until the finished design is reached (Figure 5.3).

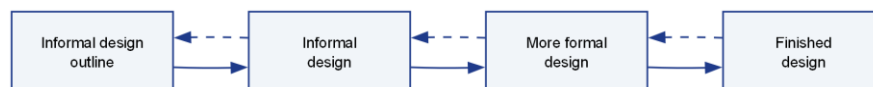


Figure 5.3. The transformation of an informal design into a detailed design.

5.1.9 Why Design Is Important

A good design is the key to a successful product. Almost two thousand years ago the Roman architect Vitruvius gave the attributes of a good design as **durability**, **utility**, and **charm**; the analogy with software is deliberate: a system, like a building, must weather change, serve its purpose, and present a coherent structure. A well-designed system is easy to implement, understandable, and reliable, and it allows the software to evolve smoothly. Without design, we risk building an unstable system: one that will fail when small changes are made, one that will be difficult to maintain, and one whose quality cannot be assessed until late in the software process. Software design must therefore contain a sufficiently complete, accurate, and precise solution to the problem to ensure that the eventual implementation is of good quality.

Three characteristics serve as a guide for the evolution of a good design:

- The design must implement all of the explicit requirements contained in the analysis model, and it must accommodate all of the implicit requirements desired by the customer.
- The design must be a readable, understandable guide for those who generate the code and for those who test and subsequently support the software.
- The design should provide a complete picture of the software, addressing its data, functional, and behavioural domains from an implementation perspective.

Of these, understandability deserves special emphasis. A good design exists to overcome the limits of human cognition: a large problem overwhelms the human mind, and a poor design makes the situation worse. A design that is difficult to understand is not merely inconvenient; it leads to an implementation full of defects and to sharply increased development and maintenance costs. Understandability is achieved through the disciplined use of abstraction and decomposition, so that an understandable design is one that is modular and whose modules are arranged in distinct layers. The stakes are economic as well as technical: roughly sixty per cent of the total effort in the life cycle of a typical product is spent on maintenance, and a design that is hard to understand multiplies that effort while producing software that is unreliable.

The reason design is emphasised so heavily, in the end, is leverage. Design sits at the point in the life cycle where a decision is cheapest to make and most expensive to reverse. A structural mistake caught during design costs a redrawn diagram; the same mistake caught after implementation costs a rebuild. This asymmetry, cheap to fix early and costly to fix late, is the practical justification for spending a disproportionate share of project effort on getting the design right.

5.2 Modularity

Modularity is the decomposition of a system into a set of separately named and addressable units, called modules, that are integrated to satisfy the requirements. It is the property that makes a large program intellectually manageable, and it is the foundation on which the rest of this chapter builds: the design strategies, the function-oriented approach, and even the object-oriented approach are all ways of producing a good decomposition. This section first fixes what a module and a modular system are, then examines the cost trade-off that decides how far to modularise, and finally treats the two forces, coupling and cohesion, by which the quality of a decomposition is judged.

5.2.1 Modularity and the Modular System

There is no single definition of the term *module*, and the source insists that this is not a problem because every one of the usual definitions is correct at its own level of granularity. The term ranges from a **FORTRAN subroutine**, through an **Ada package**, the **procedures and functions of Pascal and C**, the **C++ and Java classes**, the **Java packages**, up to a **work assignment for an individual programmer**. What all of these have in common is stated as a single sentence: a modular system consists of **well defined, manageable units with well defined interfaces among the units**.

A system is considered modular if it consists of **discrete components** such that each component can be **implemented separately**, and a **change to one component has minimal impact on the other components**. The second half of that condition is the one that carries the weight: it is easy to break a program into files, but modularity is only achieved when a change stays local.

Modularity pays for itself because it lets a large problem be attacked by the divide-and-conquer principle. If the modules interact only slightly, each one can be understood on its own, which reduces the perceived complexity of the whole. The usual illustration is a bundle of sticks: a bundle is hard to break, but the

individual sticks are easy to break one at a time. Modularity, in the words of the source, is **the single attribute of software that allows a program to be intellectually manageable**; it enhances design clarity, and design clarity in turn eases implementation, debugging, testing, documenting, and maintenance.

It is important to add what modularity is *not*. A software system cannot be made modular simply by chopping it into a set of modules. Each module must support a **well defined abstraction** and must have a **clear interface** through which it interacts with the other modules. Chopping without abstraction produces small pieces that are still tangled together, and so both **under-modularity** and **over-modularity** are to be avoided.

The properties that a well-designed modular system should exhibit are the following. Each module is a **well-defined subsystem** that is potentially useful in other applications. Each module has a **single, well-defined purpose**. Modules can be **separately compiled and stored in a library**. Modules **can use other modules**. A module **should be easier to use than to build**. And a module **should be simpler from the outside than from the inside**. Read together, these properties describe a unit that hides its internal complexity behind a small, stable interface, which is exactly the idea that the principles of abstraction and information hiding, introduced in Section 5.1.6, put to work.

5.2.2 Modularity and Software Cost

How far should a system be modularised? The answer is not "as far as possible", because modularity trades two costs against each other. As the number of modules grows, the size of each module and the effort to develop it fall, but the effort required to **integrate** the modules rises. The total cost of effort is therefore a curve with a minimum: it plots cost of effort against the number of modules and marks a **region of minimum cost** at some value M of modules (Figure 5.4).

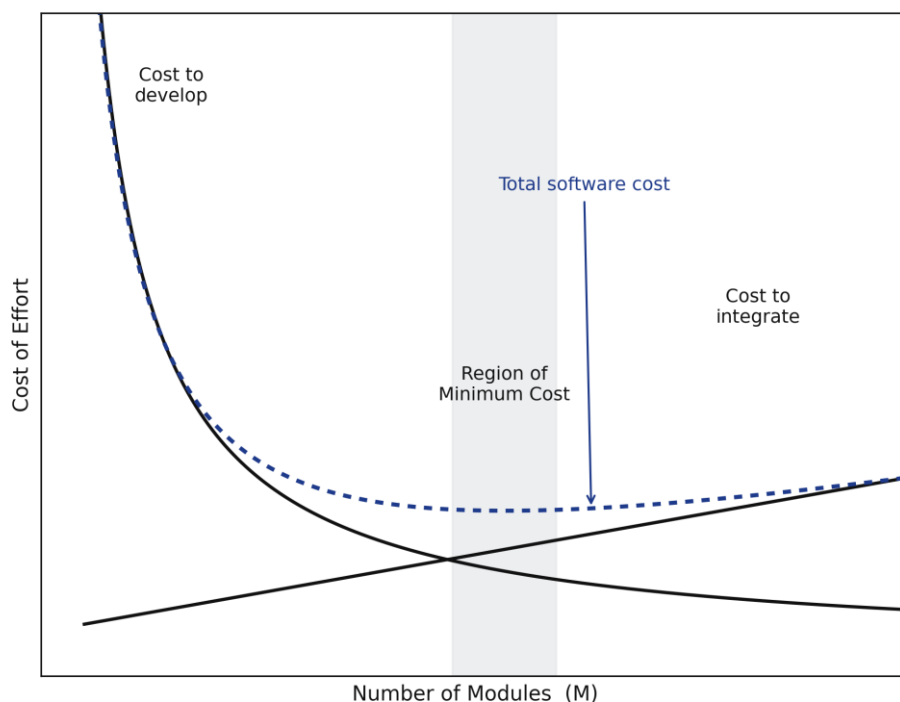


Figure 5.4. Modularity and software cost.

The argument behind the curve is worth stating because it shows why the curve cannot be pushed to zero. Perceived complexity grows faster than the sum of its parts: if $C(x)$ is the perceived complexity of a problem x

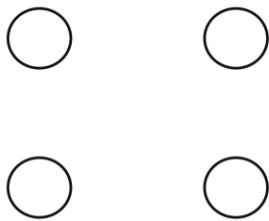
and $E(x)$ the effort to solve it, then for two problems $p1$ and $p2$ the complexity of the combined problem exceeds the sum, $C(p1 + p2) > C(p1) + C(p2)$, and the same holds for effort. This is the formal statement of divide and conquer, and it is an argument *for* modularity. But it also suggests, falsely, that subdividing indefinitely would make the effort vanish; the countervailing force is the integration cost, which is what produces the minimum at M .

The practical difficulty is that the value of M cannot be predicted with assurance. The guidance is therefore qualitative: stay in the vicinity of M , and avoid both under-modularity and over-modularity. Whether a given design method leads to an effective decomposition can be judged against five criteria: **modular decomposability** (the method provides a systematic way to break the problem into sub-problems), **modular composability** (existing reusable components can be assembled into a new system), **modular understandability** (a module can be understood as a stand-alone unit), **modular continuity** (a small change to the requirements changes individual modules rather than the whole system), and **modular protection** (an aberrant condition is confined within the module in which it occurs).

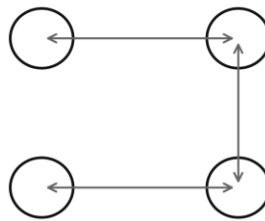
5.2.3 Module Coupling

Coupling is the **measure of the degree of interdependence between modules**. Two modules with high coupling are strongly interconnected and therefore dependent on each other; two modules with low coupling are largely independent. The source distinguishes three degrees (Figure 5.5): **uncoupled** modules have no dependencies at all and are completely independent; **loosely coupled** modules have some dependencies; and **highly coupled** modules share a great deal of dependence, for example when they make use of shared global variables.

(a) Uncoupled: no dependencies



(b) Loosely coupled: some



(c) Highly coupled: many

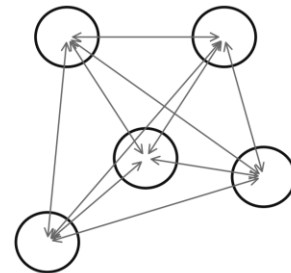


Figure 5.5. Degrees of module coupling: uncoupled, loosely coupled, and highly coupled.

Coupling is measured by the **number of interconnections between modules**: it increases as the number of calls between the modules increases, and as the amount of shared data increases. Another way to state the same thing is that the degree of coupling depends on the **interface complexity** of the modules, the number of parameters exchanged, and the complexity of those parameters, when one module invokes a function of another. The design hypothesis that follows is simple and is confirmed in practice: a design with high coupling tends to have more errors, because a fault can propagate across the connections and is hard to localise.

A good design therefore aims at low coupling, and the interfaces between modules must be specified carefully to keep it low. Four techniques achieve this: **control the number of parameters** passed between modules; **avoid passing undesired data** to the calling module; **maintain a parent/child relationship** between calling and called modules; and **pass data, not control information**.

The value of these rules is easiest to see in an example. Figure 5.6 shows two designs for editing a student record in a student information system. In the poor design, the calling module passes the student name, the student ID, the address, and the course, together with the record and an end-of-file flag; the extra items (name, address, course) are not needed by the called module and are examples of what the source calls **tramp data**, data that travels through a module without being used there. Passing this superfluous information increases overhead and reduces efficiency. In the good design, only the **student ID** is passed. The lesson generalises: the way to minimise coupling between two modules is to have them communicate using only the data they actually need.

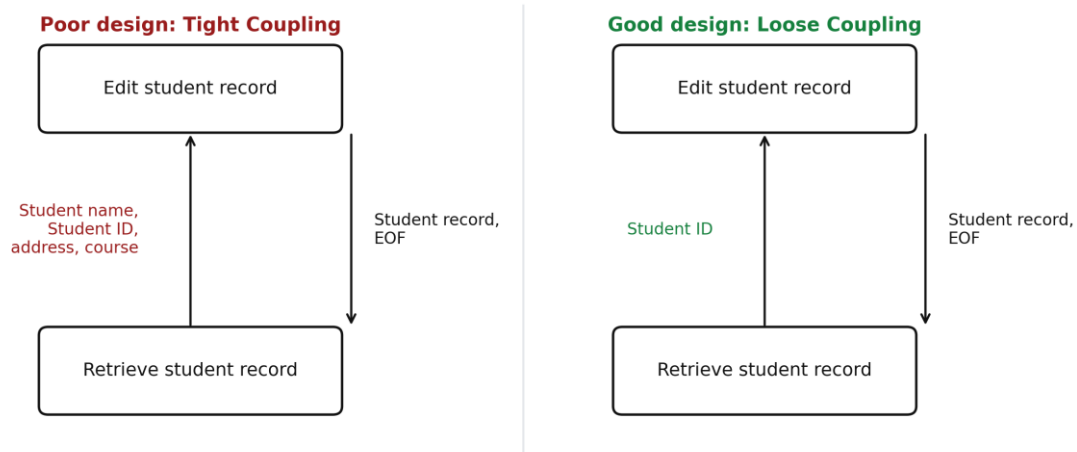


Figure 5.6. Example of coupling: editing a student record, tight versus loose.

Given two procedures A and B, there are several recognisable ways in which they can be coupled. From best to worst, the types are **data**, **stamp**, **control**, **external**, **common**, and **content** coupling (Figure 5.7).

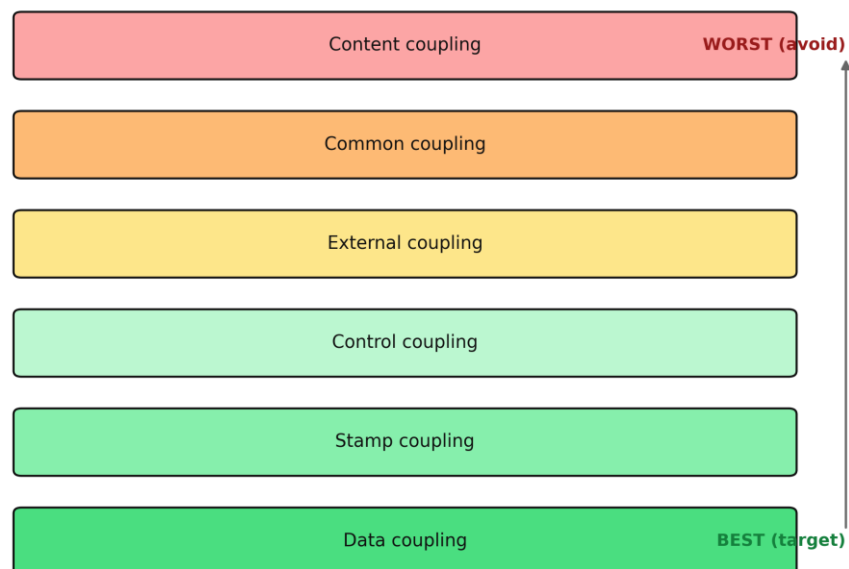


Figure 5.7. The types of module coupling, from best (data) to worst (content).

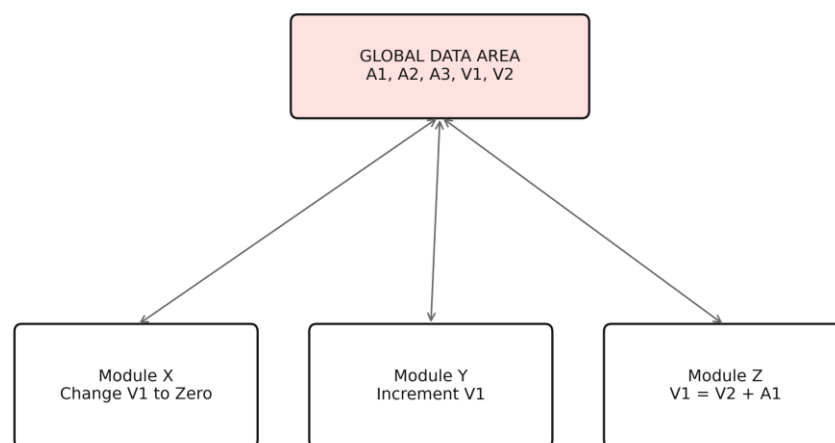
Data coupling is the best form. Two modules are data coupled if their only dependency is that they communicate by passing data, typically an elementary data item such as an integer, a float, or a character. Apart from that exchange the two modules are independent. The data item must be problem-related and must not be used for control purposes. The rule that follows is to ensure that no module communication contains tramp data, and that each module receives only what it needs.

Stamp coupling occurs when a **complete data structure**, a record in Pascal, a structure in C, an object in C++, is passed from one module to another, even though the receiving module uses only part of it. Stamp coupling typically involves tramp data. If a procedure needs only part of a structure, the calling module should pass just that part rather than the whole structure.

Control coupling exists when data from one module is used to **direct the order of instruction execution** in another. It is usually implemented by a flag that is set in one module and tested in the other. The sending module must know a great deal about the inner workings of the receiving module, which is precisely why control coupling is undesirable.

External coupling is a dependency of a module on something **external to the software**, on another module outside the software being developed, or on a particular type of hardware. It is essentially communication with external tools and devices. Such coupling is sometimes unavoidable, but it should be confined to as few modules as possible.

Common coupling exists when two modules **share data**, typically through a global data area. Global data areas are common in programming languages, and they are convenient, but they have a hidden cost: making a change to the common data means tracing back to every module that accesses that data in order to evaluate the effect of the change, and with common coupling it can be difficult to determine which module is responsible for setting a variable to a particular value. Figure 5.8 shows three modules, X, Y and Z, each reading or writing the same global variables. Common coupling is not automatically wrong, global data has its uses, but the designer must be aware of its consequences and guard against them.



All modules read/write the same global variables -> changes ripple everywhere

Figure 5.8. Example of common coupling through a shared global data area.

Content coupling is the worst form. It occurs when one module **changes the data of another**, or when **control is passed into the middle of another module**, or when the two modules **share code** through a jump

from one into the other. The source's example is a module B that branches into D even though D is supposed to be under the control of C. In languages such as C, such jumps across modules are not even possible, which is one reason modern high-level languages are preferred.

The practical consequence of the whole ladder can be summarised in a single design rule: prefer the top of the list and avoid the bottom. High coupling among modules not only makes a design difficult to understand and maintain, but also increases development effort and makes it very hard for one module to be developed independently by a different team member.

5.2.4 Module Cohesion

Cohesion is the **measure of the degree to which the elements of a module are functionally related**. Where coupling looks outward, across module boundaries, cohesion looks inward: it is, in the source's image, the **glue that keeps the module together**, and it measures the mutual affinity of the components of a module. A strongly cohesive module implements functionality that relates to a single feature of the solution and requires little or no interaction with other modules.

The design objective for cohesion is the mirror image of the objective for coupling. Since cohesion measures the strength of the relations *within* a module, we want to **maximise** it; since coupling measures the strength of the relations *between* modules, we want to **minimise** it. Stated as one goal: maximise module cohesion and minimise module coupling.

Cohesion is not a single condition but a ladder of seven levels. From weakest to strongest they are **coincidental**, **logical**, **temporal**, **procedural**, **communicational**, **sequential**, and **functional** cohesion (Figure 5.9). The definitions are best understood by asking how two operations X and Y inside the same module are related.

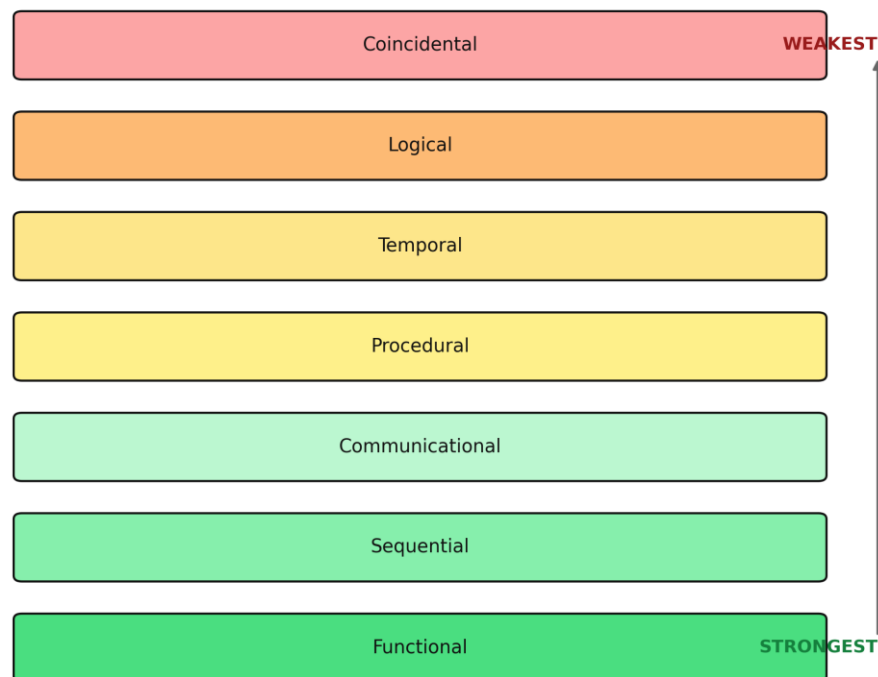


Figure 5.9. The types of module cohesion, from weakest (coincidental) to strongest (functional).

Coincidental cohesion is the weakest level. X and Y have **no conceptual relationship** other than that they share the same code. The elements come together by coincidence rather than by design, and the module typically contains a random collection of statements that happen to have been placed together. It is to be avoided as far as possible. The designs of novice programmers frequently fall into this category.

Logical cohesion occurs when the elements of a module **perform logically similar operations** but are not actually related. They appear to belong together because they fall into the same logical class of functions, for example, a module containing a set of print routines for different report types, or a set of routines that each check a different field for validity. Logical cohesion permits considerable duplication; the source's remedy is to extract the repeated logic into a single module (a `DATECHECK` module, called whenever a date check is needed) rather than to duplicate it across a class.

Temporal cohesion occurs when the elements **must be executed within the same time span**. The classic example is a start-up module that initialises memory and devices, or a module that performs all of a program's initialisation, start-up, or shut-down activities. The activities are related by *when* they happen, not by *what* they do, and that is a weak reason to keep them in one module.

Procedural cohesion occurs when the instructions, although they accomplish **different tasks**, have been combined because there is a **specific order** in which the tasks must be completed. It usually results from first charting the flow of the solution and then grouping each sequence of steps into a module. The tasks are related only by order, so such modules tend to be relatively hard to maintain. A report module that calculates the student's GPA, prints the student record, computes the cumulative GPA, and prints it is a typical case.

Communicational cohesion occurs when the elements **operate on the same input data or contribute to the same output data**. A module that uses the "student grade record" as input while calculating both the current and the cumulative GPA is an example. It is an acceptable level, though the designer might still consider splitting it into separate procedures.

Sequential cohesion occurs when the **output of one element is the input to the next** in a sequence. Addition of the marks of individual subjects into a specific format that is then used to calculate the GPA is an example: the first operation produces data that the second consumes. Because the operations are linked by a real data flow, this is a strong reason to keep them together.

Functional cohesion is the best level. X and Y are **part of a single functional task**, which is a very good reason for them to be in the same procedure. Mathematically flavoured routines such as "calculate current GPA" or "cumulative GPA" are typical. A module with functional cohesion can be described by a single simple sentence; a payroll module whose functions (compute overtime, compute work hours, compute deductions) all cooperate to generate the payslips is functionally cohesive if its entire responsibility can be stated as one sentence, "it produces the employees' payslips."

A useful practical test follows from this. To judge the cohesiveness of a module, first examine what its functions actually do, then try to describe the module's overall work in one sentence. If a **compound sentence** is needed, the module has sequential or communicational cohesion. If the description needs words such as "**first**", "**next**", "**after**", or "**then**", the module has sequential or temporal cohesion. If it needs words such as "**initialise**", "**setup**", or "**shut down**", the module has temporal cohesion.

The two forces combine into a single decisive property. A module that is **highly cohesive and has low coupling** with other modules is said to be **functionally independent**. Functional independence is the key to a good design because it brings three advantages. It gives **error isolation**: because the module interacts little with others, an error in it is unlikely to propagate, and once a failure is observed it is easier to locate the fault. It gives **scope for reuse**: a functionally independent module performs a well-defined task with few and simple

interfaces, so it can be lifted out and reused elsewhere. And it improves **understandability**: independent modules can be understood in isolation, which reduces the perceived complexity of the whole design.

5.2.5 The Relationship between Cohesion and Coupling

The essence of the design process is that a system is decomposed into parts, and the purpose of that decomposition is to make the system easier to understand and to modify. Most projects do not fail because of massive requirement changes; they fail because seemingly trivial enhancements and changes are made to a structure that cannot absorb them. If the software is not properly modularised, a host of apparently minor changes will push the project towards failure. A good design therefore achieves a clean decomposition of the problem into modules, and arranges those modules in a neat hierarchy. The rule that captures all of this is the one already stated: **the software engineer must design the modules with the goal of high cohesion and low coupling**.

The two properties are not independent targets to be chased separately; they are two sides of the same objective. A module with high cohesion and low coupling behaves like a **black box** within the structure of the system: because its functionality is well defined and its interfaces are few, the rest of the system can be described, and the module can be dealt with, purely in terms of what it does.

A familiar analogy makes the goal concrete. A computer with the "plug and play" property consists of a motherboard with slots into which add-on components can be inserted or removed without affecting the rest of the machine. This works because each add-on component provides its service in a **highly cohesive** manner and connects to the system through a small, well-defined interface, that is, with **low coupling**. Software modules designed to the same standard should be equally easy to add or remove.

Figure 5.10 shows the two objectives side by side. On the left, many interaction links run between a few modules: coupling is high and the parts are tangled. On the right, the same modules are connected by a small number of links: coupling is low and each module is a coherent unit.

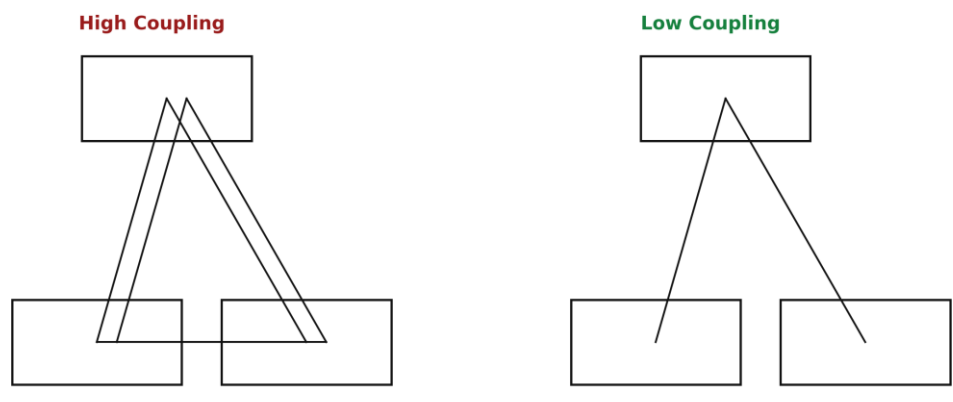


Figure 5.10. A view of cohesion and coupling: high coupling (left) versus low coupling (right).

5.2.6 Layered Arrangement of Modules

Good modularity is not only about the strength of the links; it is also about their **direction**. The control hierarchy of a design is the organisation of its modules in terms of which module calls which. In a **layered** design, the modules are arranged into several layers based on their call relationships, and a module is allowed

to call only the modules in the layer immediately below it; it should not call a module in a higher layer, nor even one in its own layer.

A layered design achieves **control abstraction**. The top-most module acts as a manager that only invokes the services of the lower-level modules; the intermediate modules offer services upward while invoking the services of the layer below; and the lowest-layer modules are the workers, which invoke no other modules and do their work entirely themselves. Such a design is easier to understand, because to understand one module it is enough to consider the modules it directly calls, and it is easier to debug, because an error in a module can affect only the modules above it, so only the lower layers need to be investigated when a failure appears. If modules may call each other arbitrarily, the design degenerates into a single layer and locating a fault becomes both difficult and time-consuming.

Several terms describe a layered structure. In a control hierarchy, a module that controls another is **superordinate** to it, and the one controlled is **subordinate** to the controller. A module B is **visible** to a module A if A directly calls B, so only the immediately lower layer is visible to a given module. **Control abstraction** is the principle that the modules of higher layers are abstracted away from the modules below them. The **depth** of a hierarchy is the number of layers, and its **width** is the overall span of control.

Two measures describe the connections. **Fan-out** is the number of modules directly controlled by a given module; a very high fan-out is a sign that the module lacks cohesion, since it is probably implementing several different functions rather than one, and a fan-out greater than about seven is a warning. **Fan-in** is the number of modules that directly invoke a given module; high fan-in is desirable because it represents code reuse.

5.3 Strategy of Design

The design of a system is not a single act; it is a process carried out under a chosen strategy. The strategy decides where the designer starts, how the decomposition proceeds, and in what order the modules are designed and implemented. Choosing it well matters because the strategy shapes the whole structure that follows, and changing strategy halfway through a design is expensive. This section explains why an explicit strategy is needed at all, and then examines the three that the source recognises: bottom-up, top-down, and hybrid.

5.3.1 The Need for a Design Strategy

A good system design strategy is one that organises the program modules **in such a way that they are easy to develop and, later, to change**. Structured design techniques exist to help developers cope with the size and complexity of programs: the analyst produces instructions for the developers describing how the code should be written and how the pieces of code should fit together to form a program. This is important for two reasons. First, even **pre-existing code**, if there is any, has to be understood, organised, and pieced together with the new code. Second, it is still common for a project team to have to **write some original code** that supports the application logic of the system rather than assembling everything from existing parts.

It is worth understanding why the older approach fell out of favour, because the reason shapes what a good strategy must do. In the early days, if design was done at all it was essentially "writing down the flowchart in words". Many people felt that flowcharts were **too detailed**: they forced the detailed decisions to be made too early, far from the specifications. The result was a sudden jump from the specification to a low-level flowchart, and that jump was itself a cause of many errors. Worse, an error in a flowchart could only be found by coding it, discovering that the code ran wrongly, diagnosing that the error was in the flowchart, and then modifying the code and recording the change. Repeated surgery of this kind could eventually produce a flowchart in which further errors could no longer be fixed, and the project could fail.

For this reason, writers of large and complex software now seldom use flowcharts for design. Instead, a family of **higher-level notations** is used, chosen so that the distance from specification to design, and from design to code, is as short as possible. These notations usually allow **several levels of design**, so that the transformation is made in many small jumps rather than in one or two massive ones. Against this background the source identifies three strategies for performing the design: the **bottom-up** approach, the **top-down** approach, and the **hybrid** approach.

5.3.2 Bottom-Up Design

The bottom-up approach starts from parts that already exist or can easily be built. A common way to begin is to identify the **modules that are required by many programs** and to collect them together in the form of a **library**. The library might hold mathematical functions, input-output functions, graphical functions, or modules for a result-preparation system such as "maintain student details", "maintain subject details", and "marks entry". These low-level modules are then combined to provide larger ones, the larger ones are combined in turn, and the process continues upward until eventually one module represents the whole of the desired program. The result is a hierarchy in which each module is **subordinate to those in which it is used** (Figure 5.11).

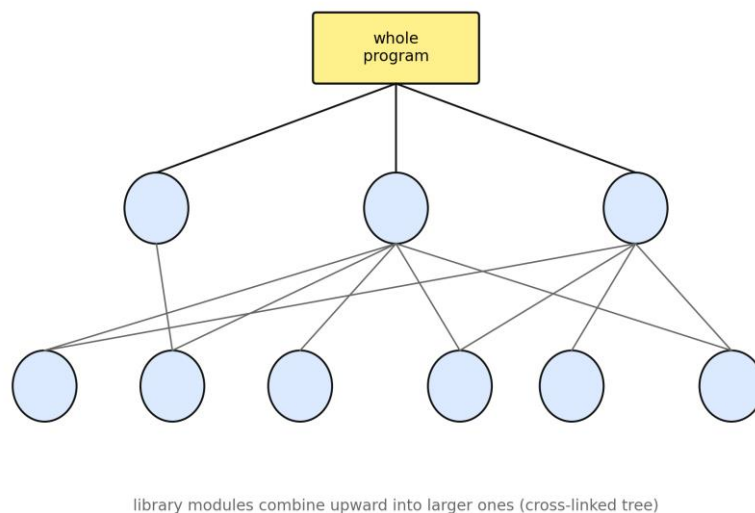


Figure 5.11. Bottom-up tree structure: library modules combine upward into larger modules.

Because the design progresses from the bottom layer upward, the method is called bottom-up design. Its principal attraction is a practical one about testing: if a module is coded soon after it is designed, there is a good chance it may have to be recoded later, but at least the coded module can be **tested and its design validated sooner** than a module whose sub-modules have not yet been designed. In other words, it allows working, tested components to exist early.

The method has, in the source's blunt phrasing, one **terrible weakness**: it requires a great deal of **intuition** to decide exactly what functionality each module should provide. If that guess is wrong, the error will not surface until a higher level, where the result is found not to meet the requirements, and the designer must then go back and redesign at a lower level. This makes bottom-up design least suitable for a system built from scratch and most suitable when the system is to be built from **an existing system**, since it begins from modules that already exist.

5.3.3 Top-Down Design

The essential idea of top-down design is that the specification is viewed as describing a **black box** for the program. The designer decides how the interior of that black box is to be constructed from **smaller black boxes**, and specifies those; the process is then repeated inside each inner black box, and continues until the black boxes are small enough to be coded directly. Operationally, a top-down design **starts by identifying the major modules of the system, decomposing them into their lower-level modules, and iterating until the desired level of detail is achieved**. The source gives this the classical name of **stepwise refinement**: starting from an abstract design, each step refines the design to a more concrete level, until a level is reached where no further refinement is needed and the design can be implemented directly.

Most design methodologies are based on this approach, and it is well suited to a project whose **specifications are clear and whose development is from scratch**. Its characteristic drawback is the mirror image of the bottom-up method's strength: if coding of a part begins soon after its design, nothing can be **tested** until all of its subordinate modules have been coded, because a high-level module depends on the lower-level modules beneath it.

Top-down refinement has a further consequence for reuse that is easy to miss. If a program is created strictly top-down, the modules become **highly specialised**: each module is used by at most one other module, its parent. Reuse, however, requires the opposite, a module that is required by several different parents. The pure top-down tree therefore has to be relaxed into a **reusable structure**, in which a shared module sits beneath more than one parent (Figure 5.12). Recognising this tension, the source adds that it is not necessary to create a program strictly top-down; if the aim is to defer the decision of what the system is to do as long as possible, it is better to structure the program **around the data** rather than around the actions it performs.

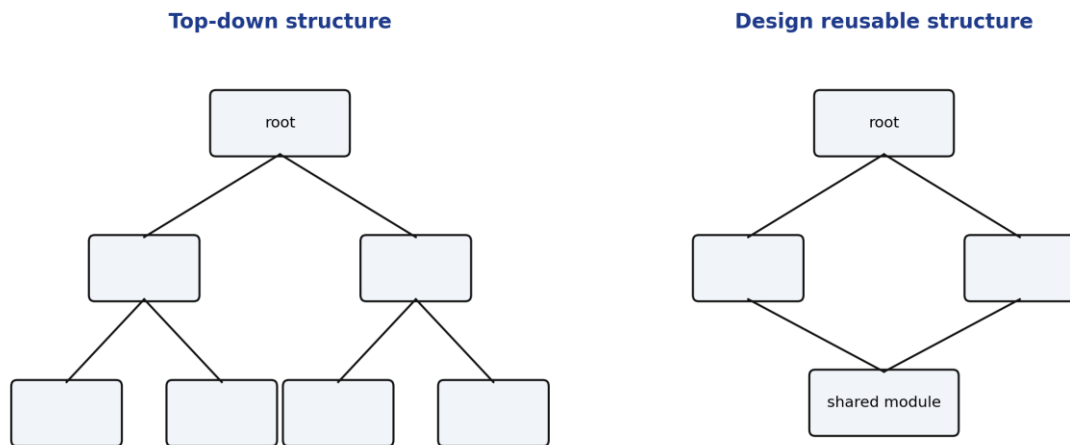


Figure 5.12. Top-down structure (left) versus the design-reusable structure (right).

5.3.4 Hybrid Design

Neither pure top-down nor pure bottom-up design is practical on a real project, and the source is explicit about why. For a **bottom-up** approach to succeed, the designer must already have a good notion of **the top toward which the design is heading**; without a good idea of the operations needed at the higher layers, it is very difficult to decide what operations the current, lower layer should support. Conversely, for a **top-down** approach to be effective, some bottom-up work is essential at the lowest design levels, for three reasons:

- it allows **common sub-modules** to be shared rather than duplicated;

- near the **bottom of the hierarchy** the intuition is simpler and the **need for bottom-up testing is greater**, because there are more modules at the low levels than at the high levels;
- it makes use of **pre-written library modules**, that is, of **reuse**.

A hybrid strategy therefore combines a top-down decomposition of the overall structure with a bottom-up use of existing, tested components at the bottom. The source notes that the hybrid approach became genuinely popular only **after reusability of modules was accepted**, and it points to standard libraries, class libraries such as the Microsoft Foundation Classes, and object-oriented concepts as the steps in that direction. The expectation is that reuse will continue to grow until internationally accepted standards for reusable modules exist.

5.3.5 Choosing among the Strategies

The three strategies differ in their starting point, their direction of travel, and their natural home. The comparison is summarised below.

Strategy	Starts from	Direction	Works well when	Main weakness
Bottom-up	Existing / library modules	Upward, composing larger units	Building from an existing system; reusable parts already exist	Needs much intuition about what each module should do; errors surface late
Top-down	The specification (black box)	Downward, stepwise refinement	Specifications are clear; development from scratch	Nothing can be tested until subordinate modules exist; modules too specialised to reuse
Hybrid	Whole structure top-down, components bottom-up	Both	Large real systems that also reuse libraries	Requires discipline to keep the two directions consistent

The practical guidance is simple. Use **top-down** when the specification is clear and the system is new. Use **bottom-up** when a good set of reusable modules already exists and the task is to assemble a system from them. Use **hybrid** for anything of realistic size: decompose the overall structure from the top to guarantee that the requirements are met, but allow the lowest levels to be assembled from existing, tested components so that reuse and early testing are not sacrificed.